Báo cáo tiến độ tuần 3

Cài đặt AES-128 trên ma trận 2D, dẫn xuất khóa PBKDF2, đệm PKCS#7, chế độ khối CBC và mã hóa tệp tin

Học phần PBL3: Mạng thông tin máy tính

GVHD: TS. Trần Thị Minh Hạnh

SVTH: Nguyễn Thái Hoàng – 106230011

Lê Trương Uyển Nhi – 106230054

Tóm tắt

Báo cáo trình bày việc mở rộng chương trình AES-128 từ phạm vi xử lý một khối 16 byte cố định ở Tuần 2 sang xử lý dữ liệu và tệp tin có kích thước bất kỳ. Mã nguồn được tổ chức lại thành lớp AES thao tác trên ma trận 2 chiều 4×4, bổ sung cơ chế sinh khóa ngẫu nhiên (CSPRNG), dẫn xuất khóa từ mật khẩu (PBKDF2-HMAC-SHA256), đệm dữ liệu PKCS#7 và chế độ hoạt động xích khối CBC. Chương trình thực hiện mã hóa, giải mã tệp tin nhị phân và tệp nén ZIP, đồng thời kiểm tra tính toàn vẹn dữ liệu khôi phục bằng mã băm SHA-256.

Từ khóa: *AES-128, Ma trận 2D, PBKDF2, PKCS#7, Chế độ CBC, Mã hóa tệp tin, SHA-256.*

1. Giới thiệu

1.1 Bài toán thực tế và mục tiêu của Tuần 3

Chương trình ở Tuần 2 mới chỉ mã hóa và giải mã đúng một khối 16 byte với khóa hex nhập thủ công. Để chuẩn bị cho bài toán truyền nhận tệp tin qua giao thức TCP Socket ở Tuần 4, hệ thống trong Tuần 3 cần giải quyết 5 vấn đề kỹ thuật được tổng hợp trong Bảng 1.

Bảng 1. Các vấn đề kỹ thuật cần giải quyết và ứng dụng trong Tuần 3

Thành phần	Tại sao phải có (Hạn chế ở Tuần 2)	Giải pháp ở Tuần 3	Ứng dụng trong chương trình
Kiến trúc lớp AES & Ma trận 2D	Mảng 1D (4*c+r) rời rạc; khóa vòng bị tính lặp lại ở mỗi khối khi xử lý tệp lớn.	Đóng gói class AES(key), lưu sẵn 11 khóa vòng; biểu diễn State bằng ma trận 4×4 theo cột s[col][row].	Tăng tốc khi mã hóa hàng nghìn khối liên tiếp; tách biệt tầng lõi mật mã với tầng xử lý tệp.
Sinh khóa & Dẫn xuất PBKDF2	Người dùng khó nhớ khóa 16 byte (32 ký tự hex); mật khẩu ngắn dễ bị dò bằng từ điển.	Dùng os.urandom(16) cho khóa ngẫu nhiên; dùng PBKDF2-HMAC-SHA256 kèm Salt 16B (100.000 vòng) cho mật khẩu.	Cho phép mã hóa tệp bằng mật khẩu văn bản (--password) và làm chậm tấn công vét cạn.
Đệm dữ liệu PKCS#7	Lõi AES chỉ nhận khối đúng 16 byte, trong khi văn bản và tệp thực tế có độ dài bất kỳ.	Bổ sung từ 1 đến 16 byte đệm có giá trị bằng số byte đệm thêm; kiểm tra khi gỡ đệm (unpad).	Chuẩn hóa mọi chuỗi văn bản và tệp nhị phân về bội số của 16 byte mà không làm hỏng dữ liệu gốc.
Chế độ xích khối CBC	Mã hóa từng khối độc lập (ECB) làm các khối bản rõ giống nhau sinh ra bản mã giống nhau (lộ mẫu).	XOR mỗi khối bản rõ với bản mã liền trước (khối đầu XOR với vector khởi tạo IV 16B ngẫu nhiên).	Ẩn quy luật lặp lại của dữ liệu; cùng một tệp mã hóa nhiều lần luôn tạo bản mã khác nhau.

Thành phần	Tại sao phải có (Hạn chế ở Tuần 2)	Giải pháp ở Tuần 3	Ứng dụng trong chương trình
Đóng gói tệp .enc & SHA-256	Chưa có định dạng lưu trữ IV, Salt và cơ chế kiểm tra tệp sau giải mã.	Ghi Header (PWD_MAGIC + Salt) và IV ở đầu tệp .enc; băm SHA-256 trước và sau giải mã.	Mã hóa/giải mã tệp .txt, .pdf, .docx, .zip; xác nhận khôi phục đúng từng byte.

1.2 Phân công nhiệm vụ thực hiện

Bảng 2. Phân công nhiệm vụ và đóng góp của nhóm trong Tuần 3

Thành viên	Nhiệm vụ thực hiện	Đóng góp
Nguyễn Thái Hoàng 106230011	Cài đặt lớp AES ma trận 2D, đệm PKCS#7, chế độ CBC và mô-đun mã hóa/giải mã tệp, tệp nén ZIP.	50%
Lê Trương Uyển Nhi 106230054	Cài đặt sinh khóa CSPRNG, dẫn xuất khóa PBKDF2, bộ kiểm thử unittest (13 test method) và viết báo cáo.	50%

2. Tái cấu trúc lõi AES-128 sang ma trận 2D (Lớp AES)

• **Lý do chuyển đổi và ứng dụng:** Ở Tuần 2, các phép biến đổi SubBytes, ShiftRows, MixColumns và AddRoundKey đã được trình bày trên mảng 1 chiều. Sang Tuần 3, khi mã hóa tệp tin gồm nhiều khối 16 byte, nếu gọi hàm mở rộng khóa ở từng khối sẽ gây lãng phí tính toán. Nhóm đóng gói thuật toán vào lớp AES(key) để khởi tạo trước 11 khóa vòng (self.round_keys) một lần duy nhất, đồng thời chuyển State sang danh sách 2 chiều 4 cột × 4 hàng (state[cột][hàng]) tham khảo từ boppreh/aes [3] và sách The Design of Rijndael [2].

• **Tối ưu phép biến đổi trên ma trận 2D:** Với cấu trúc state[cột][hàng], phép trộn cột mix_single_column thao tác trực tiếp trên từng danh sách cột state[i]. Phép nghịch đảo inv_mix_columns chỉ cần bước tiền xử lý 4 byte trên mỗi cột rồi gọi lại hàm mix_columns, giúp giảm số phép nhân đa thức trong GF(2^8).

Khung 1. Ánh xạ chuỗi 16 byte sang ma trận 2D (state[cột][hàng]) và hàm InvMixColumns trong aes.py

```
# Ánh xạ 16 byte b0..b15 vào 4 cột: state[0]=[b0,b1,b2,b3], ...,
state[3]=[b12,b13,b14,b15]
def bytes2matrix(text: bytes) -> list[list[int]]:
    return [list(text[i:i + 4]) for i in range(0, len(text), 4)]

def matrix2bytes(matrix: list[list[int]]) -> bytes:
    return bytes(b for col in matrix for b in col)

def inv_mix_columns(s: list[list[int]]) -> None:
    for i in range(4):
        u = xtime(xtime(s[i][0] ^ s[i][2]))
        v = xtime(xtime(s[i][1] ^ s[i][3]))
        s[i][0] ^= u; s[i][1] ^= v; s[i][2] ^= u; s[i][3] ^= v
    mix_columns(s)
```

Để kiểm chứng kiến trúc ma trận 2D mới cho kết quả trung gian trùng khớp với chuẩn FIPS 197 [1], Bảng 3 tổng hợp trạng thái State qua từng bước của Vòng 1 với vector chuẩn Phụ lục C.1 (Plaintext P = 00112233445566778899aabbccddeeff, Key K = 000102030405060708090a0b0c0d0e0f).

Bảng 3. Trạng thái State qua các bước của Vòng 1 (hiển thị theo 4 hàng của ma trận 4×4)

1. Sau AddRoundKey (Vòng 0)	2. Sau SubBytes (Vòng 1)	3. Sau ShiftRows (Vòng 1)	4. Sau MixColumns (Vòng 1)	5. Sau AddRoundKey (Vòng 1)
00 40 80 c0 10 50 90 d0 20 60 a0 e0 30 70 b0 f0	63 09 cd ba ca 53 60 70 b7 d0 e0 e1 04 51 e7 8c	63 09 cd ba 53 60 70 ca e0 e1 b7 d0 8c 04 51 e7	5f 57 f7 1d 72 f5 be b9 64 bc 3b f9 15 92 29 1a	89 85 2d cb d8 5a 18 12 10 ce 43 8f e8 68 d8 e4

3. Phương pháp thực hiện các mô-đun mở rộng

3.1 Sinh khóa ngẫu nhiên (CSPRNG) và dẫn xuất khóa từ mật khẩu (PBKDF2)

• **Tại sao phải có:** AES-128 yêu cầu khóa K có độ dài đúng 16 byte (128 bit). Trong thực tế, người dùng thường đặt mật khẩu văn bản (passphrase) có độ dài tùy ý. Nếu chỉ ghép thêm ký tự hoặc băm SHA-256 một lần, mật khẩu dễ bị dò tìm bằng bảng băm tính sẵn (rainbow table) hoặc vét cạn tốc độ cao.

• **Nguyên lý và ứng dụng:** Chương trình cung cấp hai cơ chế: (1) Hàm generate_key() lấy 16 byte từ nguồn ngẫu nhiên hệ điều hành os.urandom(16) khi cần sinh khóa tự động; (2) Hàm derive_key() kết hợp mật khẩu UTF-8 với 16 byte muối ngẫu nhiên (Salt) và lặp 100.000 vòng băm HMAC-SHA256 theo chuẩn NIST SP 800-132 [6]. Salt giúp hai lần mã hóa với cùng một mật khẩu vẫn sinh ra hai khóa K hoàn toàn khác nhau.

Khung 2. Sơ đồ dẫn xuất khóa PBKDF2 và cài đặt trong aes.py

```
Sơ đồ:  Mật khẩu (UTF-8) + Salt ngẫu nhiên (16B) ──[ PBKDF2-HMAC-SHA256 (100.000 vòng)
]──> Khóa K (16B)

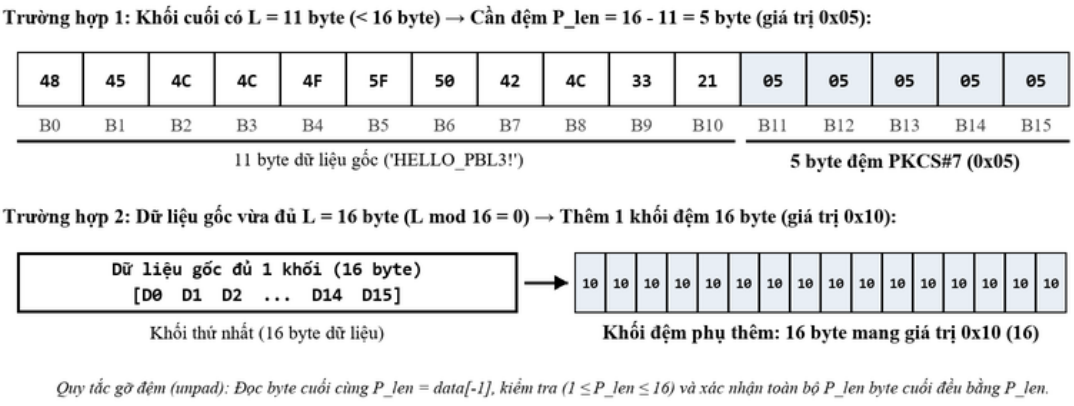
def generate_key(length: int = 16) -> bytes:
    return os.urandom(length)

def derive_key(password: str | bytes, salt: bytes | None = None, iterations: int =
100_000) -> tuple[bytes, bytes]:
    if isinstance(password, str):
        password = password.encode("utf-8")
    if salt is None:
        salt = os.urandom(16)
    key = hashlib.pbkdf2_hmac("sha256", password, salt, iterations, dklen=16)
    return key, salt
```

3.2 Cơ chế đệm dữ liệu PKCS#7 (Padding & Unpadding)

• **Tại sao phải có:** Hàm mã hóa khối của AES chỉ xử lý được các khối tròn 16 byte, trong khi văn bản hoặc tệp tin có kích thước L bất kỳ. Nếu chỉ điền các byte 0x00 vào cuối khối thiếu, bên giải mã sẽ không phân biệt được đâu là byte 0x00 thuộc nội dung tệp nhị phân gốc và đâu là byte đệm.

• **Nguyên lý và ứng dụng:** Theo chuẩn RFC 5652 (PKCS#7) [4], số byte cần đệm là P_len = 16 - (L mod 16), với $1 \leq P_len \leq 16$, và mỗi byte đệm đều mang giá trị đúng bằng P_len (Hình 1). Khi L đã chia hết cho 16, thuật toán vẫn nối thêm một khối 16 byte mang giá trị 0x10 để việc gỡ đệm không bị nhập nhằng. Hàm unpad() đồng thời giúp phát hiện sớm trường hợp giải mã sai khóa hoặc dữ liệu bị hỏng khi các byte cuối không thỏa quy tắc PKCS#7.



Hình 1. Minh họa quy tắc đệm PKCS#7 cho khối lẻ (11 byte) và khối chẵn (16 byte)

Khung 3. Cài đặt hàm thêm đệm pad() và kiểm tra gỡ đệm unpad() trong aes.py

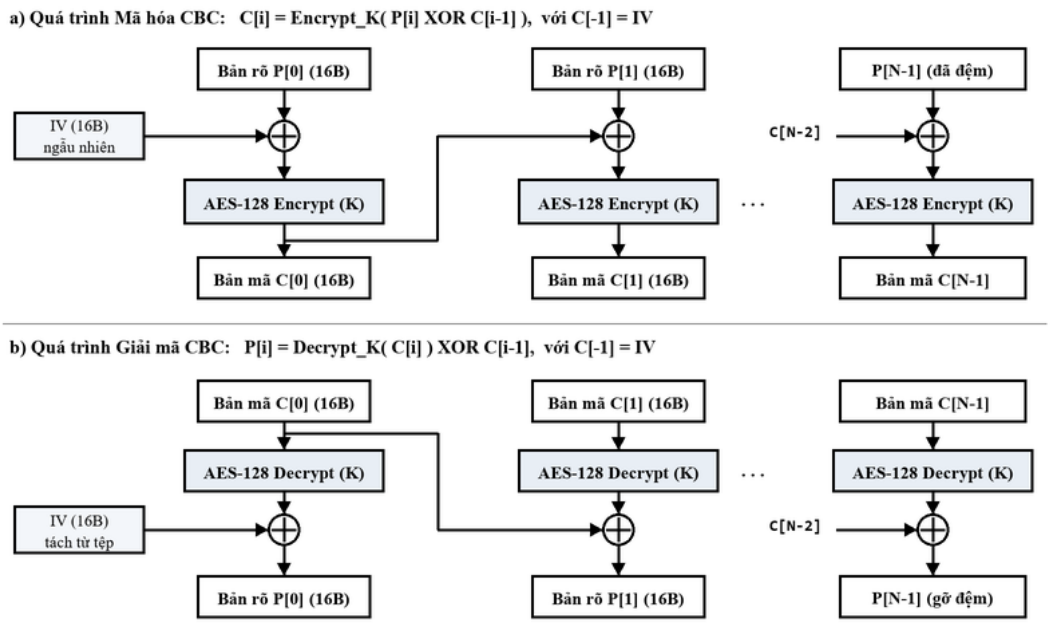
```
def pad(data: bytes, block_size: int = 16) -> bytes:
    padding_len = block_size - (len(data) % block_size)
    return data + bytes([padding_len] * padding_len)

def unpad(data: bytes, block_size: int = 16) -> bytes:
    if not data or len(data) % block_size != 0:
        raise ValueError("Dữ liệu cần unpad không hợp lệ (không phải bội số của
block_size)")
    padding_len = data[-1]
    if padding_len < 1 or padding_len > block_size:
        raise ValueError("Padding PKCS#7 không hợp lệ: giá trị byte đệm ngoài phạm vi 1-
16")
    if data[-padding_len:] != bytes([padding_len] * padding_len):
        raise ValueError("Padding PKCS#7 bị hỏng hoặc sai giá trị kiểm tra")
    return data[:-padding_len]
```

3.3 Chế độ hoạt động xích khối CBC (Cipher Block Chaining)

• **Tại sao phải có:** Khi mã hóa tệp tin gồm nhiều khối bằng chế độ ECB (mã hóa độc lập từng khối), các khối bản rõ giống nhau luôn tạo ra các khối bản mã giống nhau, làm lộ cấu trúc lặp lại của dữ liệu gốc.

• **Nguyên lý và ứng dụng:** Theo NIST SP 800-38A [5], chế độ CBC ghép xích các khối bằng phép XOR trước khi đưa vào lõi AES (Hình 2). Khối đầu tiên P[0] được XOR với Vector khởi tạo ngẫu nhiên IV (16 byte); từ khối thứ i trở đi, P[i] được XOR với bản mã C[i-1] liền trước. Khi giải mã, kết quả qua lõi decrypt_block(C[i]) được XOR ngược lại với C[i-1] để khôi phục P[i]. Nhờ sinh IV mới cho mỗi lần mã hóa, các khối bản rõ trùng nhau sẽ cho ra bản mã hoàn toàn khác nhau.



Hình 2. Sơ đồ ghép xích khối trong quá trình mã hóa (a) và giải mã (b) của chế độ CBC

Khung 4. Cài đặt phương thức encrypt_cbc() và decrypt_cbc() trong lớp AES (aes.py)

```
def encrypt_cbc(self, plaintext: bytes, iv: bytes | None = None) -> tuple[bytes, bytes]:
    if iv is None:
        iv = os.urandom(16)
    padded = pad(plaintext)
    ciphertext_blocks, prev = [], iv
    for offset in range(0, len(padded), 16):
        block = padded[offset:offset + 16]
        prev = self.encrypt_block(bytes(a ^ b for a, b in zip(block, prev)))
        ciphertext_blocks.append(prev)
    return iv, b"".join(ciphertext_blocks)

def decrypt_cbc(self, ciphertext: bytes, iv: bytes) -> bytes:
    plaintext_blocks, prev = [], iv
    for offset in range(0, len(ciphertext), 16):
        block = ciphertext[offset:offset + 16]
        decrypted = self.decrypt_block(block)
        plaintext_blocks.append(bytes(a ^ b for a, b in zip(decrypted, prev)))
        prev = block
    return unpad(b"".join(plaintext_blocks))
```

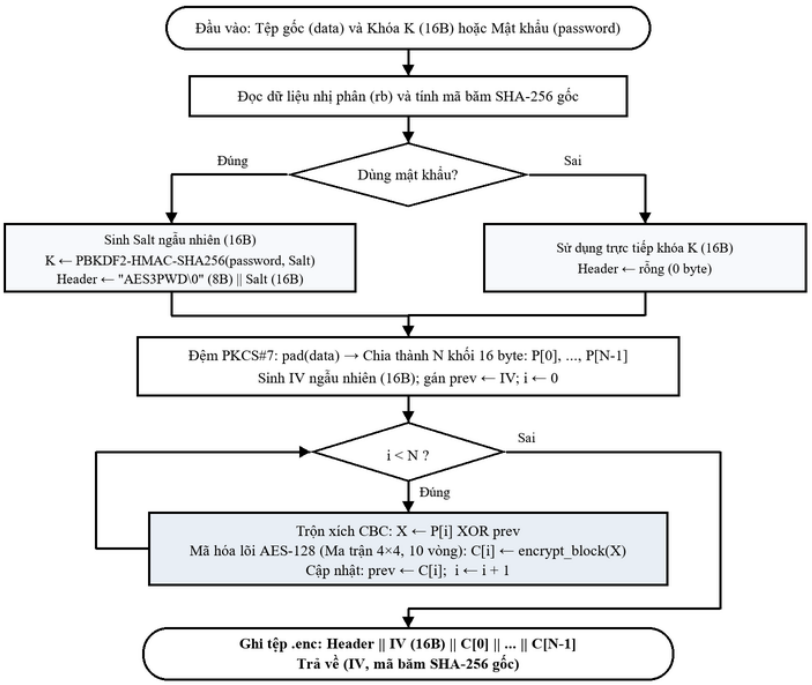
3.4 Quy trình mã hóa, giải mã tệp tin / ZIP và cấu trúc tệp .enc

• **Tại sao phải đóng gói Header và kiểm tra SHA-256:** Mọi định dạng tệp (.txt, .pdf, .docx, .zip) đều là chuỗi byte nhị phân. Để quá trình giải mã diễn ra độc lập chỉ từ một tệp .enc duy nhất, chương trình cần lưu kèm IV (và Salt nếu mã hóa bằng mật khẩu) ngay ở phần đầu tệp. Đặc biệt với tệp nén .zip, chỉ cần sai lệch 1 byte sau giải mã sẽ làm hỏng bảng mục lục Central Directory và không thể giải nén; do đó mã băm SHA-256 được tính trên cả tệp gốc và tệp khôi phục để đối chiếu.

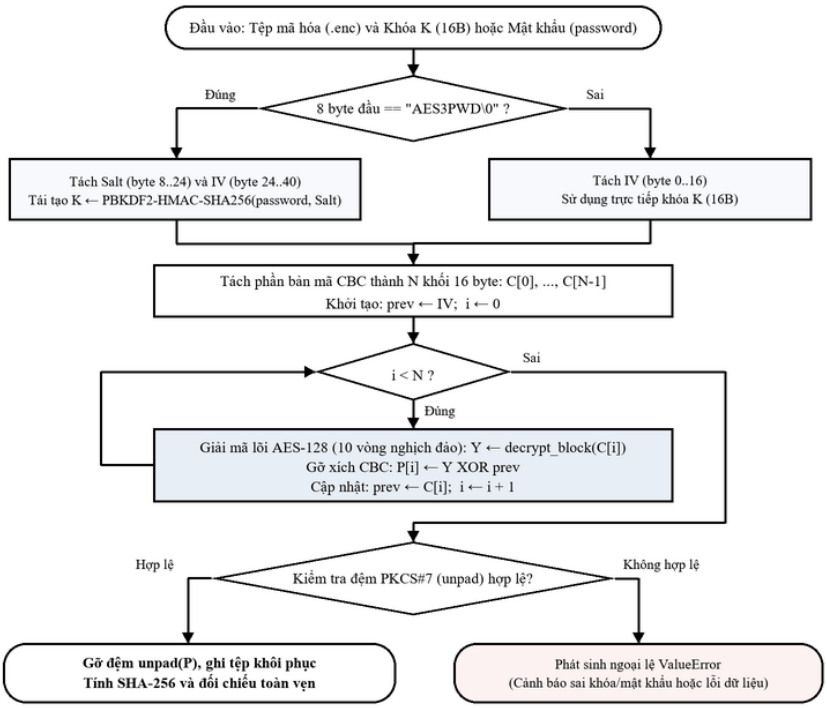
Bảng 4. Cấu trúc khung dữ liệu của tệp mã hóa (.enc) theo hai chế độ khóa

Phương thức khóa	**Cấu trúc các trường byte trong tệp .enc**	**Cách bóc tách khi giải mã**
Khóa trực tiếp 16B (--key)	`[ IV (16 byte) ] || [ Bản mã CBC (N × 16 byte) ]`	Đọc 16 byte đầu làm IV; phần còn lại đưa vào decrypt_cbc().
Mật khẩu văn bản (--password)	`[ PWD_MAGIC "AES3PWD\0" (8B) ] || [ Salt (16B) ] || [ IV (16B) ] || [ Bản mã CBC ]`	Kiểm tra 8 byte đầu khớp "AES3PWD\0", lấy byte 8..24 làm Salt để tái tạo khóa K qua PBKDF2, lấy byte 24..40 làm IV.

Toàn bộ luồng xử lý kết hợp các mô-đun của Tuần 3 được thể hiện trên Hình 3 (quy trình mã hóa) và Hình 4 (quy trình giải mã).



Hình 3. Lưu đồ quy trình mã hóa dữ liệu và tệp tin ở Tuần 3



Hình 4. Lưu đồ quy trình giải mã và kiểm tra toàn vẹn tệp tin ở Tuần 3

4. Thực nghiệm, kiểm thử và giới hạn kỹ thuật

4.1 Thực nghiệm mã hóa và giải mã tệp nén ZIP

Nhóm thực hiện kịch bản demo tự động (python demo.py --demo-zip) trên tệp nén sample_archive.zip chứa 2 tệp thành phần (baocao_pbl3.txt và data/students.csv).

Khung 5. Kết quả chạy thực nghiệm mã hóa tệp nén ZIP (python demo.py --demo-zip)

```
1. Tệp ZIP gốc       : sample_archive.zip (373 bytes)
   SHA-256 gốc       : ed77c38073bd1126c19d0572161b4fd5a864b99a5fce78bc2cd12b07d3cce994
2. Tệp mã hóa CBC    : sample_archive.zip.enc (400 bytes = 16B IV + 384B Ciphertext)
3. Tệp sau giải mã   : sample_archive_recovered.zip (373 bytes)
   SHA-256 khôi phục : ed77c38073bd1126c19d0572161b4fd5a864b99a5fce78bc2cd12b07d3cce994
4. Đối chiếu SHA-256 : PASS (Trùng khớp mã băm và giải nén đọc lại đúng 2 tệp bên trong)
```

Phân tích kích thước tệp: Tệp ZIP gốc dài L = 373 byte. Do 373 mod 16 = 5, hàm pad() chèn thêm P_len = 16 - 5 = 11 byte mang giá trị 0x0B để tạo thành 384 byte (tương ứng 24 khối 16 byte). Khi ghép thêm 16 byte IV ở đầu tệp, tệp .enc có kích thước đúng 400 byte.

4.2 Kiểm thử tự động bằng thư viện unittest

Bộ kiểm thử trong tệp test_aes.py gồm 13 test method (chạy bằng lệnh python -m unittest -v), đối chiếu từ lõi toán học đến xử lý tệp tin thực tế. Kết quả được tổng hợp trong Bảng 5.

Bảng 5. Kết quả kiểm thử tự động trong test_aes.py (13 test method)

Hạng mục kiểm thử	Dữ liệu và kịch bản kiểm tra	Kết quả
Lõi AES-128 & Khóa vòng	Vector chuẩn FIPS 197 Phụ lục C.1, 11 khóa vòng K0..K10 và 4 vector NIST AES_Core128.	PASS (5/5 vector)

Hạng mục kiểm thử	Dữ liệu và kịch bản kiểm tra	Kết quả
Chế độ CBC chuẩn NIST	Đối chiếu trực tiếp 4 khối bản mã CBC (64 byte) với vector chuẩn NIST SP 800-38A Phụ lục F.2.1.	PASS (4/4 khối)
Đệm và gỡ đệm PKCS#7	Kiểm tra độ dài biên 0, 1, 2, 15, 16, 17, 31, 32, 100, 255 byte và bắt lỗi ValueError khi byte đệm sai.	PASS (10/10 độ dài)
Sinh khóa & PBKDF2	Kiểm tra tính tiền định khi cùng (password, Salt) và tính phân tách khóa khi khác Salt hoặc mật khẩu.	PASS
Tính ngẫu nhiên của IV	Mã hóa hai lần cùng một bản rõ dưới chế độ CBC phải sinh ra hai cặp (IV, bản mã) khác nhau.	PASS
Mã hóa tệp & Tệp nén ZIP	Mã hóa/giải mã tệp nhị phân bằng khóa trực tiếp và mật khẩu; từ chối khi sai mật khẩu; giải nén lại tệp ZIP.	PASS (13/13 OK)

4.3 Giới hạn kỹ thuật của chương trình hiện tại

• **Chưa nhúng mã xác thực thông điệp (MAC) vào tệp .enc:** Chế độ CBC chỉ bảo vệ tính bí mật. Khi giải mã với mật khẩu sai, khối cuối tạo ra dữ liệu ngẫu nhiên và có xác suất khoảng 1/256 (~0,39%) byte cuối tình cờ bằng 0x01, khiến hàm unpad() không báo lỗi. Ở Tuần 4, nhóm sẽ bổ sung HMAC-SHA256 theo mô hình Encrypt-then-MAC để xác thực toàn vẹn trước khi giải mã.

• **Đọc toàn bộ tệp vào bộ nhớ RAM:** Hàm encrypt_file() hiện đọc toàn bộ nội dung tệp bằng read_bytes(), phù hợp cho tệp vừa và nhỏ. Khi truyền tệp qua mạng ở Tuần 4, chương trình sẽ được tách thành từng khung dữ liệu (chunk) kích thước cố định.

5. Kết luận và hướng phát triển Tuần 4

Trong Tuần 3, nhóm đã hoàn thành việc tái cấu trúc AES-128 sang lớp đối tượng thao tác trên ma trận 2D, cài đặt dẫn xuất khóa PBKDF2, đệm dữ liệu PKCS#7, chế độ xích khối CBC và ứng dụng mã hóa/giải mã cho các định dạng tệp tin nhị phân, tệp nén ZIP. Toàn bộ 13 test method đối chiếu với chuẩn NIST đều vượt qua. Trong Tuần 4, nhóm sẽ tích hợp mô-đun này vào ứng dụng Client – Server truyền nhận tệp tin qua giao thức TCP Socket kết hợp xác thực HMAC-SHA256.

Tài liệu tham khảo

[1] National Institute of Standards and Technology (NIST), Advanced Encryption Standard (AES), FIPS PUB 197, 2001 (cập nhật 2023). https://doi.org/10.6028/NIST.FIPS.197-upd1

[2] Joan Daemen, Vincent Rijmen, The Design of Rijndael: AES - The Advanced Encryption Standard, Springer, 2002.

[3] Lucas Boppre Niehues (boppreh), Pure Python AES implementation, GitHub Repository: https://github.com/boppreh/aes (MIT License).

[4] R. Housley, Cryptographic Message Syntax (CMS) - Section 6.3: Content-encryption process (PKCS#7 Padding), RFC 5652, 2009.

[5] NIST Special Publication 800-38A, Recommendation for Block Cipher Modes of Operation: Methods and Techniques, 2001.

[6] NIST Special Publication 800-132, Recommendation for Password-Based Key Derivation: Part 1: Storage Applications, 2010.